LeetCode 2653 - Sliding Subarray Beauty

ALGORITHM DOCUMENTATION & OPTIMIZATION GUIDE

1. Problem Understanding

For every subarray (window) of size k, we have to find:

- The x-th smallest negative integer in that window.
- If the window contains fewer than x negative numbers, return 0.

> **Notice carefully:**
> - We only care about negative numbers.
> - Positive numbers and 0 never become the answer.

Example

Input: `nums = [1, -1, -3, -2, 3]`, $k = 3$, $x = 2$

Window 1: `[1, -1, -3]`

- Negative numbers: `[-1, -3]`
- Sorted: `[-3, -1]`
- 2nd smallest: `-1`

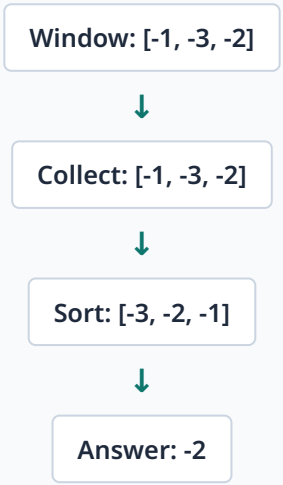
Window 2: `[-1, -3, -2]`

- Sorted negatives: `[-3, -2, -1]`
- 2nd smallest: `-2`

2. First Thought (Brute Force)

For every window:

- Collect negative numbers.
- Sort them.
- Return x-th element.



Metric	Complexity / Detail
Time Complexity	$O(n \times k \log k)$
Status	Too slow because $n = 10^5$

3. Important Observation

Windows overlap.

Example:
Window 1: `[1, -1, -3]` → Window 2: `[-1, -3, -2]`

Only:

- One element leaves
- One element enters

So rebuilding every window is unnecessary. This tells us **Sliding Window** should be used.

4. Biggest Observation (The Trick)

Constraint: $-50 \le nums[i] \le 50$

Normally, finding the x-th smallest element requires:

- `multiset`
- Balanced BST
- Fenwick Tree
- Segment Tree

But here, negative values are only `-50, -49, ..., -1`. There are only **50 different negative numbers**. Instead of storing numbers, we store their frequency.

5. Frequency Array

We create:

```
vector<int> freq(50);
```

It stores only negative numbers using the following index mapping:

Number	-50	-49	-48	...	-2	-1
Index	0	1	2	...	48	49

Formula: `index = value + 50;`

Example:
`value = -3`
`index = -3 + 50 = 47`
So, `freq[47]` means the **frequency of -3**.

6. Why +50?

Array indices cannot be negative, so we shift every value:

- `-50 → 0`
- `-49 → 1`
- ...
- `-1 → 49`

When we need the original number:

Formula: `value = index - 50;`

Example: `index = 47` → *47 - 50 = -3*

7. Building First Window

Suppose the window is `[2, -3, -1]`. Positive numbers are ignored.

Frequency becomes:

- `-3 → 1`
- `-1 → 1`

```
if (nums[i] < 0)
    freq[nums[i] + 50]++;
```

Notice that `2` is never stored.

8. Sliding Window

Suppose:

- Old window: `[2, -3, -1]`
- New window: `[-3, -1, -5]`

Left element (2): Positive. Ignore because it was never added.

Right element (-5): Negative. Increase its frequency.

Now frequency correctly represents `[-3, -1, -5]`.

If the left element is negative:

Example: `[-4, -2, 3]` → `[-2, 3, -1]`

Remove it using:

```
freq[-4 + 50]--;
```

because `-4` left the window.

9. Why do we ignore positives?

Because the answer can never be `5, 7, 0, 9`, etc. The problem asks only for the **x-th smallest negative number**. Positive numbers never affect the answer, so we never store them.

10. How do we find the x-th smallest negative?

This is the most important idea.

Suppose the window is `[-3, -1, -3, -2]`.

Number	Frequency
-3	2
-2	1
-1	1

The sorted negatives are `[-3, -3, -2, -1]`. We never actually build this sorted array. Instead, we scan from `-50 → -49 → ... → -3 → -2 → -1` and maintain a counter `cnt`.

Suppose $x = 3$:

- Scan `-50`: `cnt = 0`
- ...
- Scan `-3`: `cnt += 2` → `cnt = 2`
- Scan `-2`: `cnt += 1` → `cnt = 3`

Now `cnt ≥ x`, so the **Answer is -2**. This matches the sorted array precisely.

11. How are duplicates handled?

Example A: Window `[-5, -5, -2, -1]` | Sorted: `[-5, -5, -2, -1]` | $x = 2$ (2nd smallest)

- Frequency: `-5 → 2`, `-2 → 1`, `-1 → 1`
- Scan `-5`: `cnt = 2`. Since `2 ≥ x`, return **-5** (Correct).

Example B: Window `[-5, -5, -2, -2]` | Sorted: `[-5, -5, -2, -2]` | $x = 4$ (4th smallest)

- Scan `-5`: `cnt = 2`
- Scan `-2`: `cnt = 4`. Returns **-2** (Correct again).

Duplicates work automatically because `cnt += freq[i];` adds all occurrences of that number.

12. Why do we scan from -50 to -1?

Because $-50 < -49 < ... < -1$. This is already the sorted order. So scanning the frequency array is equivalent to scanning the sorted list of negative numbers.

13. What if there are fewer than x negatives?

Example: Window `[-3, 5, 6]` | Frequency: `-3 → 1` | Suppose $x = 2$
Scan finishes with `cnt = 1`. It never reaches 2, so we **Return 0** exactly as required.

14. Overall Algorithm

Step 1: Build the frequency of negative numbers for the first window.

Step 2: Call `getBeauty(freq, x)` to compute the answer.

Step 3: Slide the window.

Remove element leaving the window:

```
if (nums[i - k] < 0)
    freq[nums[i - k] + 50]--;
```

Add new element entering the window:

```
if (nums[i] < 0)
    freq[nums[i] + 50]++;
```

Again call `getBeauty()`. Repeat for every window.

15. Intuition Behind getBeauty()

Think of the frequency array as a compressed sorted array. Instead of storing `[-5, -5, -3, -2, -2, -1]`, we store:

- `-5 → 2`
- `-3 → 1`
- `-2 → 2`
- `-1 → 1`

Scanning `cnt += freq[i];` is exactly the same as expanding this list mentally.

Number	Frequency	Cumulative Count
-5	2	2
-4	0	2
-3	1	3
-2	2	5
-1	1	6

If $x = 4$, then $2 < 4$, $3 < 4$, but $5 \geq 4$. So the answer is **-2**.

16. Time Complexity

- Building first window: $O(k)$
- Each slide: $O(1)$
- Finding answer: Only **50 iterations**.

Overall Time Complexity: $O(k + (n - k) \times 50) \approx O(n)$

17. Space Complexity

- Frequency Array: 50 integers.

Overall Space Complexity: $O(50) \approx O(1)$

Final Intuition (Remember This)

This is a **Sliding Window + Frequency Counting** problem. For each window, we maintain the frequencies of only the negative numbers. Since the values are limited to -50 through -1, the frequency array is a compressed representation of the sorted negative numbers. By scanning it from -50 to -1 and accumulating frequencies, we effectively walk through the sorted negatives (including duplicates) until we reach the x-th one. This avoids sorting every window and gives an overall O(n) solution with constant extra space.